

a critical section. Consider the code snippet below:

```
int tickets[MAX_ROWS][MAX_COLS];
pthread_mutex_t tickets_lock;

void BookTicket(int row, int column)
{
    pthread_mutex_lock(&tickets_lock);
    tickets[row][column].status = 'booked';
    pthread_mutex_unlock(&tickets_lock);
}
```

Here, we have a single mutex lock *tickets_lock* that protects the entire tickets table. At a time, only one thread can hold the lock and hence only one ticket can be booked, even though there can be two people requesting tickets such as (1,1) and (10,10) which are totally unrelated tickets. Since we use a single lock to protect the entire tickets table, we prevent concurrency among independent accesses to unrelated parts of the tickets table.

The above code snippet is an example of coarse-grained locking. Now we can reduce the granularity of the lock, by associating a unique lock with each row and each column in the cinema hall. Consider the code snippet given below:

```
int tickets[MAX_ROWS][MAX_COLS];
pthread_mutex_t row_lock;
pthread_mutex_t column_lock;

void BookTicket(int row, int column)
{
    pthread_mutex_lock(&row_lock);
    pthread_mutex_lock(&column_lock);
    tickets[row][column].status = 'booked';
    pthread_mutex_unlock(&column_lock);
    pthread_mutex_unlock(&row_lock);
}
```

In this snippet, we can allow two threads that are trying to book tickets (i1,j1) and (i2,j2) such that i1 is not equal to i2 and j1 is not equal to j2, to book tickets concurrently. This increases the concurrency compared to the coarse grained locking case when we used a single lock for the whole tickets table.

Still, there is concurrency to be gained by using finer grained locks, since a ticket is uniquely specified by a row and column, and as long as the two threads access two tickets in separate locations [row1][column1] and [row2][column2], where either 'row1' is not equal to 'row2' or 'column1' is not equal to 'column2', the threads can book the tickets concurrently.

In order to allow this, we need to associate a lock with each unique seat location, namely, each element of the tickets matrix. This is demonstrated in the code snippet below:

```
int tickets[MAX_ROWS][MAX_COLS];
pthread_mutex_t ticket_locks[MAX_ROWS][MAX_COLS];

void BookTicket(int row, int column)
{
    pthread_mutex_lock(&ticket_locks[row][column]);
    tickets[row][column].status = 'booked';
    pthread_mutex_unlock(&ticket_locks[row][column]);
}
```

However, note that we end up using as many mutex locks as there are elements in the tickets matrix. This can lead to excessive lock overhead, since for each 'tickets' element access, we have to access one mutex lock.

Earlier, when we used coarse grained locks, a single mutex was used to protect multiple 'tickets' elements. This resulted in lowered concurrency, but also reduced the locking overhead.

Hence, the lock granularity one needs to choose for an application is often dependent on the access patterns of the data shared by the threads. The locking overhead associated with fine grained locks needs to be traded off against the increased concurrency achieved by fine grained locking.

Note that lock granularity has a major impact on lock contention, which we shall discuss in our next column.

Takeaway problem for this month

As I mentioned before, we keep open the takeaway problem from last month since there were no correct solutions. Based on the hints given in this column, please do send me your solutions to the maximal sub-rectangle problem.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with solutions and feedback to sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

References

- [en.wikipedia.org/wiki/Lock_\(computer_science\)](http://en.wikipedia.org/wiki/Lock_(computer_science))
- en.wikipedia.org/wiki/Spinlock

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.